friend. The sooner you make peace with the fact that your quality of life will drastically increase when you unit test your code, the better. Seriously. We'll make one in a moment.

Let's see the interface Rails created for us.

```
$ bin/rails server
```

Go to your browser and open http://localhost:3000/high_scores, now we can create new high scores (55,160 on Space Invaders!)

1.4 `rails console`

The `console` command lets you interact with your Rails application from the command line. On the underside, `rails console` uses IRB, so if you've ever used it, you'll be right at home. This is useful for testing out quick ideas with code and changing data server-side without touching the website.
You can also use the alias "c" to invoke the console: `rails c`.

You can specify the environment in which the `console` command should operate.

```
$ bin/rails console staging
```

If you wish to test out some code without changing any data, you can do that by invoking `rails console --sandbox`.

```
$ bin/rails console --sandbox
Loading development environment in sandbox (Rails 5.0.0)
Any modifications you make will be rolled back on exit
irb(main):001:0>
```

1.4.1 The app and helper objects

Inside the `rails console` you have access to the `app` and `helper` instances.
With the `app` method you can access url and path helpers, as well as do requests.

```
>> app.root_path
=> "/"

>> app.get _
Started GET "/" for 127.0.0.1 at 2014-06-19 10:41:57 -0300
...
```

With the `helper` method it is possible to access Rails and your application's helpers.

```
>> helper.time_ago_in_words 30.days.ago
=> "about 1 month"

>> helper.my_custom_helper
=> "my custom helper"
```

1.5 `rails dbconsole`

`rails dbconsole` figures out which database you're using and drops you into whichever command line interface you would use with it (and figures out the command line parameters to give to it, too!). It supports MySQL, PostgreSQL, SQLite and SQLite3.
You can also use the alias "db" to invoke the dbconsole: `rails db`.

1.6 `rails runner`

`runner` runs Ruby code in the context of Rails non-interactively. For instance:

```
$ bin/rails runner "Model.long_running_method"
```

You can also use the alias "r" to invoke the runner: `rails r`.

You can specify the environment in which the `runner` command should operate using the `-e` switch.

```
$ bin/rails runner -e staging "Model.long_running_method"
```

You can even execute ruby code written in a file with runner.

```
$ bin/rails runner lib/code_to_be_run.rb
```